

for_code[]

[Fique conectado]



Acesse para treinar

db-fiddle.com/



Material da
Capacitação

andradasdev.github.io/sql/



Siga a for_code no
Instagram

instagram.com/forcodeufrj

2026-08-27

[Capacitação de MySQL]



db-fiddle.com/ andradasdev.github.io/sql/ instagram.com/forcodeufrj

for_code[]

[Capacitação de MySQL]

Fundamentos de banco de dados relacional

Ronivaldo D. Andrade - Diretor de Projetos – 27 de agosto de 2026

2026-08-27

[Capacitação de MySQL]

- []

[Capacitação de MySQL]

Ronivaldo D. Andrade - Diretor de Projetos – 27 de agosto de 2026

00:00 — Abertura (5 min para o bloco todo)

Boas-vindas. Diga seu nome e uma frase sobre por que este tema importa para a liga.

Faça agora: peça para todo mundo já abrir o navegador. A URL do phpMyAdmin vem no bloco 2 — quanto antes eles abrirem, menos tempo se perde lá.

[Apresentação]



Ronivaldo D. Andrade

**Estagiário de Análise e Desenvolvimento de Sistemas
na DGOM — Marinha do Brasil
Diretor de Projetos da for_code
Matemática Aplicada — UFRJ**

ronidomingues.ard@gmail.com

[linkedin.com/in/ronidomingues](https://www.linkedin.com/in/ronidomingues) · github.com/ronidomingues

2026-08-27

[Capacitação de MySQL]



Estagiário de Análise e Desenvolvimento de Sistemas
na DGOM — Marinha do Brasil
Diretor de Projetos da for_code
Matemática Aplicada — UFRJ

00:00 — ainda na abertura

Quarenta segundos, no máximo. O que importa aqui não é o currículo: é deixar claro que você usa isto no trabalho, e não só na sala de aula.

Faça agora: dite o e-mail em voz alta e diga que aceita dúvida depois da capacitação — isso costuma destravar quem não pergunta na hora.

[O trato de hoje]

Você vai sair daqui sabendo

- Criar um banco e uma tabela
- Inserir, consultar, alterar e apagar dados
- Filtrar com **WHERE**
- Ligar duas tabelas com **JOIN**

O que NÃO cabe em 2 horas

- Normalização formal
- Índices e performance
- Transações e backup

Em 2 horas ninguém vira DBA.

Mas todo mundo sai capaz de montar o banco do primeiro projeto.

2026-08-27

[Capacitação de MySQL]

Você vai sair daqui sabendo

O que NÃO cabe em 2 horas

WHERE JOIN

Em 2 horas ninguém vira DBA.

00:01

Seja explícito sobre o que fica de fora — isso evita a pergunta “mas e o Docker?” no meio do bloco 3.

Frase-âncora: “o mínimo necessário para o primeiro projeto real”.

[Como as 2 horas vão passar]

Bloco	Assunto	Início	Tempo
0	Abertura	00:00	5 min
1	Conceitos: dado, SGBD, relacional	00:05	15 min
2	Ambiente: todo mundo conectado	00:20	15 min
3	Criando banco e tabela	00:35	20 min
—	Pausa	00:55	5 min
4	Inserindo e consultando	01:00	25 min
5	Alterando e apagando	01:25	10 min
6	Ligando tabelas com JOIN	01:35	20 min
7	Erros comuns e encerramento	01:55	5 min

Dos 120 minutos, cerca de 75 são de mão na massa.

2026-08-27

[Capacitação de MySQL]

Bloco	Assunto	Início	Tempo
0	Abertura	00:00	5 min
1	Conceitos: dado, SGBD, relacional	00:05	15 min
2	Ambiente: todo mundo conectado	00:20	15 min
3	Criando banco e tabela	00:35	20 min
—	Pausa	00:55	5 min
4	Inserindo e consultando	01:00	25 min
5	Alterando e apagando	01:25	10 min
6	Ligando tabelas com JOIN	01:35	20 min
7	Erros comuns e encerramento	01:55	5 min

Dos 120 minutos, cerca de 75 são de mão na massa.

00:02

Deixe este slide na tela enquanto fala. Combine a pausa em voz alta — as pessoas se organizam melhor sabendo quando ela vem.

Controle de tempo: se um bloco atrasar, o corte sai do *bônus de agregação* no bloco 6. O JOIN não se corta.

2026-08-27

[Capacitação de MySQL]

BLOCO 1

00:05 · 15 min

BLOCO 1

Do que estamos falando

00:05 · 15 min

[Dado, informação e banco]

Dado

Valor bruto. Sozinho não diz nada.

'Ana'
2024-03-11
7

Informação

Dado dentro de um contexto.

“Ana ingressou na liga em 11/03/2024 e está no 7º período.”

Banco de dados

Coleção organizada de dados, guardada de forma persistente e estruturada.

2026-08-27

[Capacitação de MySQL]

00:06

Use um exemplo da própria liga em vez do da tela, se tiver um à mão.

Pergunte: “onde vocês guardam dados hoje?” — a resposta quase sempre é planilha. Guarde essa resposta: ela volta no bloco 6, quando o JOIN mostra o que a planilha não faz.

Dado	Informação	Banco de dados
Valor bruto. Sozinho não diz nada.	Dado dentro de um contexto.	Coleção organizada de dados, guardada de forma persistente e estruturada.
'Ana' 2024-03-11 7		

[SGBD: quem realmente cuida dos dados]

O **SGBD** é o software servidor que gerencia o banco. Em inglês, DBMS.

- Grava e lê os dados em disco
- **Garante a integridade** — não deixa entrar dado que viola suas regras
- Controla o acesso concorrente — várias pessoas escrevendo sem corromper nada
- Controla as permissões de cada usuário

MySQL · PostgreSQL · SQL Server · Oracle · SQLite

2026-08-27

[Capacitação de MySQL]

00:09

O ponto que precisa colar: **o SGBD não é o banco**. Ele é o programa que cuida do banco.

Analogia que funciona: o banco de dados é o acervo; o SGBD é o bibliotecário. Você não entra no acervo — você pede ao bibliotecário.

O **SGBD** é o software servidor que gerencia o banco. Em inglês, DBMS.

■ **Garante a integridade**

MySQL · PostgreSQL · SQL Server · Oracle · SQLite

[Cuidado com esta frase]

“A única forma de falar com um SGBD é por SQL.”

Isso é falso.

- A conversa real acontece por um **protocolo de rede binário** — o MySQL escuta na porta **3306**
- O SQL é a linguagem que **tráfega dentro** desse protocolo
- O MySQL também expõe o **X DevAPI**, uma API de CRUD sem SQL
- Quando você usa um **ORM** (Prisma, Eloquent, Hibernate), escreve objetos — o ORM traduz para SQL antes de enviar

SQL é a interface **padrão e dominante** — não a única.

2026-08-27

[Capacitação de MySQL]

00:11

Slide de correção conceitual. Vale gastar um minuto porque essa frase aparece em muito material ruim da internet.

Não aprofunde X DevAPI nem ORM — só planta a semente. Se perguntarem, responda que ORM é assunto de outra capacitação.

“A única forma de falar com um SGBD é por SQL.”
Isso é falso.

- **3306** protocolo de rede binário
- **X DevAPI**
- **ORM**

SQL é a interface **padrão e dominante** — não a única.

[Modelo relacional]

Edgar F. Codd, 1970. Os dados vivem em **tabelas**.

```
TABELA membro
```

```
+-----+-----+-----+-----+
| id | nome          | email          | periodo |
+-----+-----+-----+-----+
| 1 | Ana Souza     | ana@liga.edu.br | 7 |
| 2 | Bruno Lima    | bruno@liga.edu.br | 3 |
+-----+-----+-----+-----+
```

- **Linha** (registro): uma ocorrência do mundo real
- **Coluna** (campo): uma característica dela, com **tipo fixo**
- **Chave**: o que liga uma tabela a outra — daí “relacional”

2026-08-27

[Capacitação de MySQL]

00:14

Aponte na tela: linha na horizontal, coluna na vertical. Parece óbvio para você, não é para quem nunca viu.

“**Tipo fixo**” é a diferença central para uma planilha: na planilha, alguém digita “sete” na coluna de período e ninguém reclama. Aqui o banco recusa.

Edgar F. Codd, 1970. Os dados vivem em **tabelas**.

```
TABELA membro
+----+-----+-----+-----+
| id | nome          | email          | periodo |
+----+-----+-----+-----+
| 1 | Ana Souza     | ana@liga.edu.br | 7 |
| 2 | Bruno Lima    | bruno@liga.edu.br | 3 |
+----+-----+-----+-----+
```

- Linha
- Coluna
- Chave

tipo fixo

[SQL não é um bloco só]

Sigla	Para quê	Comandos
DDL	Define a estrutura	CREATE ALTER DROP
DML	Manipula os dados	INSERT UPDATE DELETE
DQL	Consulta os dados	SELECT
DCL	Controla permissões	GRANT REVOKE
TCL	Controla transações	COMMIT ROLLBACK

Hoje: DDL (uma vez) + DML e DQL (o tempo todo).

2026-08-27

[Capacitação de MySQL]

Sigla	Para quê	Comandos
DDL	Define a estrutura	CREATE ALTER DROP
DML	Manipula os dados	INSERT UPDATE DELETE
DQL	Consulta os dados	SELECT
DCL	Controla permissões	GRANT REVOKE
TCL	Controla transações	COMMIT ROLLBACK

Hoje: DDL (uma vez) + DML e DQL (o tempo todo).

00:17

Não decore a tabela com eles. O que precisa ficar é a distinção entre **mexer na estrutura** e **mexer nos dados**.

Frase: “criar tabela você faz uma vez no projeto; inserir e consultar, a aplicação faz milhares de vezes por dia”.

Fim do bloco 1 — devem ser 00:20. Se passou de 00:22, acelere o bloco 2 pulando o slide de conflito de portas.

2026-08-27

[Capacitação de MySQL]

BLOCO 2

00:20 · 15 min

BLOCO 2

Todo mundo conectado

00:20 · 15 min

[Você não vai instalar nada]

O MySQL e o phpMyAdmin rodam na máquina do instrutor. Você acessa por uma URL, no navegador.

URL _____
Usuário aluno
Senha aluno

Na primeira visita aparece uma tela de aviso do túnel.
Clique em **Visit Site** e siga.

2026-08-27

[Capacitação de MySQL]

00:20 — o bloco mais arriscado da aula

Escreva a **URL no quadro** ou cole no chat da turma. Ela muda a cada vez que o túnel sobe.

Avis da tela de aviso **ANTES** de alguém perguntar — senão viram 5 minutos de “professor, deu uma tela estranha”.

Circule pela sala. Não avance enquanto todo mundo não estiver dentro.

O MySQL e o phpMyAdmin rodam na máquina do instrutor. Você acessa por uma URL, no navegador.

URL _____
Usuário aluno
Senha aluno

Na primeira visita aparece uma tela de aviso do túnel.
Clique em **Visit Site** e siga.

[phpMyAdmin: onde vamos digitar]

Um **cliente SQL** é o programa por onde você conversa com o servidor.

- Ele abre a conexão
- Envia os comandos
- Mostra o resultado em tabela

O phpMyAdmin é uma **aplicação web** — não é programa instalado. Por isso basta o navegador.

Outros clientes

MySQL Workbench

DBeaver

HeidiSQL

mysql no terminal

2026-08-27

[Capacitação de MySQL]

00:24

Mostre na tela: a barra lateral (bancos) e a **aba SQL** — é nela que tudo acontece hoje.

Deixe claro que servidor e cliente são **programas separados**, e podem estar em máquinas diferentes. É o que está acontecendo agora: o servidor é o seu, o cliente é o navegador deles.

Um **cliente SQL** é o programa por onde você conversa com o servidor.

Outros clientes

O phpMyAdmin é uma **aplicação web** — não é programa instalado. Por isso basta o navegador.

[Mão na massa #1: crie o seu banco]

Abra a aba SQL e execute:

```
SELECT VERSION(), NOW();
```

Voltou versão e data? Está conectado. Agora crie o seu banco — troque SEUNOME pelo seu primeiro nome:

```
CREATE DATABASE liga_SEUNOME CHARACTER SET utf8mb4;  
USE liga_SEUNOME;
```

liga_ana liga_joao liga_carla

2026-08-27

[Capacitação de MySQL]

Abra a aba SQL e execute:

```
SELECT VERSION(), NOW();
```

Voltou versão e data? Está conectado. Agora crie o seu banco — troque SEUNOME pelo seu primeiro nome:

```
CREATE DATABASE liga_SEUNOME CHARACTER SET utf8mb4;  
USE liga_SEUNOME;
```

liga_ana liga_joao liga_carla

00:27 — primeira digitação da turma

Diga em voz alta: minúsculas, sem acento, sem espaço. liga_joao, nunca liga_João.

Por que cada um tem o seu: o servidor é um só. Se todos usassem o mesmo banco, o CREATE TABLE do segundo já daria erro e um DELETE apagaria o dado de todos.

Erro esperado: ERROR 1044 = o nome não começa com liga_. É proposital, protege o meu banco de demonstração.

[Por que cada um tem o seu banco]

Um servidor só, a turma inteira dentro.

- Se todos usassem o mesmo banco, o segundo `CREATE TABLE` já falharia
- Um `DELETE` de qualquer um apagaria o dado de todos
- O login `aluno` só tem permissão em bancos `liga_*`

```
✓ CREATE DATABASE liga_ana
× USE cap    (banco do instrutor)
× CREATE DATABASE bagunca
```

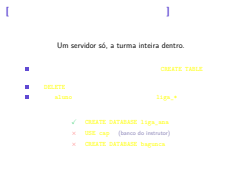
2026-08-27

[Capacitação de MySQL]

00:31

Slide curto, de fechamento do bloco. Se o tempo estiver bom, faça a demonstração ao vivo: tente `USE cap` no seu phpMyAdmin logado como `aluno` e mostre o `ERROR 1044` na tela.

Fim do bloco 2 — devem ser 00:35.



2026-08-27

[Capacitação de MySQL]

BLOCO 3

00:35 · 20 min

BLOCO 3

Criando a estrutura

00:35 · 20 min

[A ordem nunca muda]

1. CREATE DATABASE	->	criar o banco	DDL
2. USE	->	selecionar o banco	feito UMA vez
3. CREATE TABLE	->	criar a tabela	
	v		
4. INSERT	->	inserir dados	DML/DQL
5. SELECT	->	consultar	o tempo todo
6. UPDATE	->	alterar	
7. DELETE	->	remover	

Sempre existe estrutura antes do dado.

2026-08-27

[Capacitação de MySQL]

1. CREATE DATABASE	->	criar o banco	DDL
2. USE	->	selecionar o banco	feito UMA vez
3. CREATE TABLE	->	criar a tabela	
	v		
4. INSERT	->	inserir dados	DML/DQL
5. SELECT	->	consultar	o tempo todo
6. UPDATE	->	alterar	
7. DELETE	->	remover	

Sempre existe estrutura antes do dado.

00:35

A confusão mais comum aqui é achar que criar tabela e inserir dado são a mesma coisa. **Não são.**

Criar tabela: uma vez, no projeto. Inserir: milhares de vezes, pela aplicação, todo dia.

[Tipos de dados]

Tipo

INT
TINYINT
VARCHAR(n)
TEXT
DATE
DATETIME
TIMESTAMP
BOOLEAN
DECIMAL(p,s)
FLOAT

Guarda

Inteiro
Inteiro pequeno
Texto até *n*
Texto longo
AAAA-MM-DD
Data e hora
Data e hora com fuso
Verdadeiro ou falso
Número exato
Número aproximado

Use para

IDs, contadores
Período, idade
Nome, e-mail
Descrições
Data de ingresso
Agendamentos
criado_em
Sinalizadores
Dinheiro
Nunca dinheiro

Regra de ouro: dinheiro em DECIMAL. FLOAT é aproximado — 0,1 + 0,2 pode não dar 0,3.

2026-08-27

[Capacitação de MySQL]

00:38

Não leia a tabela inteira. Destaque três coisas:

1. VARCHAR é o que vocês mais vão usar;
2. dinheiro é DECIMAL, nunca FLOAT — isso já custou processo a muita empresa;
3. o tipo é o que impede “banana” de entrar na coluna de período.

Tipo	Guarda	Use para
INT	Inteiro	IDs, contadores
TINYINT	Inteiro pequeno	Período, idade
VARCHAR(n)	Texto até <i>n</i>	Nome, e-mail
TEXT	Texto longo	Descrições
DATE	AAAA-MM-DD	Data de ingresso
DATETIME	Data e hora	Agendamentos
TIMESTAMP	Data e hora com fuso	Sinalizadores
BOOLEAN	Verdadeiro ou falso	Dinheiro
DECIMAL(p,s)	Número exato	Nunca dinheiro
FLOAT	Número aproximado	

Regra de ouro: dinheiro em DECIMAL. FLOAT é aproximado — 0,1 + 0,2 pode não dar 0,3.

[Mão na massa #2: as duas tabelas]

```
CREATE TABLE curso (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(80) NOT NULL UNIQUE  
) ENGINE=InnoDB;
```

```
CREATE TABLE membro (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(160) NOT NULL UNIQUE,  
  periodo TINYINT NOT NULL,  
  data_ingresso DATE NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  curso_id INT NULL  
) ENGINE=InnoDB;
```

2026-08-27

[Capacitação de MySQL]

```
CREATE TABLE curso (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(80) NOT NULL UNIQUE  
) ENGINE=InnoDB;  
  
CREATE TABLE membro (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(120) NOT NULL,  
  email VARCHAR(160) NOT NULL UNIQUE,  
  periodo TINYINT NOT NULL,  
  data_ingresso DATE NOT NULL,  
  ativo BOOLEAN NOT NULL DEFAULT TRUE,  
  curso_id INT NULL  
) ENGINE=InnoDB;
```

00:42 — segunda digitação, a mais longa da aula

Dê tempo de verdade aqui. São 15 linhas. Circule pela sala.

ENGINE=InnoDB é o que permite chave estrangeira no bloco 6. O motor antigo, MyISAM, aceita a declaração e *ignora em silêncio*.

Erro esperado: **ERROR 1046** = esqueceu o **USE**.

[As restrições são a qualidade do dado]

PRIMARY KEY

Identifica cada linha. Não repete, não aceita **NULL**.
Uma por tabela

AUTO_INCREMENT

O MySQL gera o número sozinho

NOT NULL

Campo obrigatório

UNIQUE

Não pode repetir. Pode haver várias por tabela

DEFAULT x

Valor assumido se o **INSERT** não informar

FOREIGN KEY

Liga esta tabela a outra (bloco 6)

NULL não é zero e não é texto vazio — é “**não informado**”.

2026-08-27

[Capacitação de MySQL]

PRIMARY KEY Identifica cada linha. Não repete, não aceita **NULL**.
AUTO_INCREMENT Gera por tabela.
NOT NULL O MySQL gera o número sozinho.
UNIQUE Campo obrigatório.
DEFAULT x Não pode repetir. Pode haver várias por tabela.
FOREIGN KEY Valor assumido se o **INSERT** não informar.
Liga esta tabela a outra (bloco 6).
NULL não é zero e não é texto vazio — é “não informado”.

00:48

PK vs UNIQUE é a dúvida garantida. A diferença: só existe uma PK por tabela e ela não aceita **NULL**; **UNIQUE** pode ter várias e aceita **NULL**.

Plante o **NULL** agora — no bloco 4 eles vão tentar **WHERE curso_id = NULL** e não vai funcionar.

[Conferindo o que você criou]

```
SHOW TABLES;  
DESCRIBE membro;  
SHOW CREATE TABLE membro;
```

Confira que as duas tabelas apareceram antes de irmos para a pausa.

2026-08-27

[Capacitação de MySQL]

```
SHOW TABLES;  
DESCRIBE membro;  
SHOW CREATE TABLE membro;
```

Confira que as duas tabelas apareceram antes de irmos para a pausa.

00:52

Checkpoint obrigatório: **ninguém entra na pausa sem as duas tabelas criadas.** Quem estiver travado, resolva agora — depois da pausa o ritmo acelera e ele não recupera.

Fim do bloco 3 — devem ser 00:55.

Pausa

5 minutos

Deixe o phpMyAdmin aberto.

2026-08-27

[Capacitação de MySQL]

Pausa

Deixe o phpMyAdmin aberto.

00:55 — 5 minutos, cronometrados

Aproveite para: conferir se alguém ficou para trás no **CREATE TABLE**, e deixar o **INSERT** do próximo slide já copiado para colar.

Retome em 01:00 mesmo que falte gente.

2026-08-27

[Capacitação de MySQL]

BLOCO 4

01:00 · 25 min

BLOCO 4

Inserindo e consultando

01:00 · 25 min

[CRUD: as quatro operações]

Create **INSERT** DML

Read **SELECT** DQL

Update **UPDATE** DML

Delete **DELETE** DML

Todo sistema que você escrever na vida
faz essas quatro coisas.

2026-08-27

[Capacitação de MySQL]

01:00

Slide de mapa. 30 segundos, não mais.

Vale dizer: “de agora até o fim da aula, tudo é uma dessas quatro”.

Create **INSERT** DML
Read **SELECT** DQL
Update **UPDATE** DML
Delete **DELETE** DML

Todo sistema que você escrever na vida
faz essas quatro coisas.

[Mão na massa #3: povoando as tabelas]

```
INSERT INTO curso (nome) VALUES
('Engenharia de Software'), ('Ciencia da Computacao'),
('Sistemas de Informacao'), ('Engenharia de Producao');
```

```
INSERT INTO membro (nome, email, periodo, data_ingresso,
curso_id)
VALUES
('Ana Souza', 'ana@liga.edu.br', 7, '2024-03-11', 1),
('Bruno Lima', 'bruno@liga.edu.br', 3, '2025-02-20', 2),
('Carla Dias', 'carla@liga.edu.br', 5, '2024-08-05', 1),
('Diego Rocha', 'diego@liga.edu.br', 9, '2023-09-14', 3),
('Elisa Prado', 'elisa@liga.edu.br', 1, '2026-03-02', NULL);
```

Repare: **id** e **ativo** não foram informados.

2026-08-27

[Capacitação de MySQL]

```
CREATE TABLE curso (nome) VALUES
('Engenharia de Software'), ('Ciencia da Computacao'),
('Sistemas de Informacao'), ('Engenharia de Producao');

INSERT INTO membro (nome, email, periodo, data_ingresso,
curso_id)
VALUES
('Ana Souza', 'ana@liga.edu.br', 7, '2024-03-11', 1),
('Bruno Lima', 'bruno@liga.edu.br', 3, '2025-02-20', 2),
('Carla Dias', 'carla@liga.edu.br', 5, '2024-08-05', 1),
('Diego Rocha', 'diego@liga.edu.br', 9, '2023-09-14', 3),
('Elisa Prado', 'elisa@liga.edu.br', 1, '2026-03-02', NULL);

Repare: id e ativo não foram informados.
```

01:01

Três regras para dizer em voz alta:

1. texto entre **aspas simples**, número sem aspas;
2. data sempre **'AAAA-MM-DD'**;
3. a ordem dos valores segue a ordem das colunas listadas.

Repare na Elisa: **curso_id** é **NULL**. Ela volta no bloco 6 e é ela que faz o **LEFT JOIN** fazer sentido. Não deixe apagarem.

[SELECT: a estrutura completa]

```
SELECT    colunas
FROM      tabela
WHERE     condicao           -- filtra LINHAS
ORDER BY  coluna [ASC|DESC] -- ordena
LIMIT     n;                -- limita a quantidade
```

```
SELECT * FROM membro;
SELECT nome, email FROM membro;
SELECT nome AS aluno, periodo AS sem FROM membro;
```

2026-08-27

[Capacitação de MySQL]

```
SELECT colunas
FROM   tabela
WHERE  condicao -- filtra LINHAS
ORDER BY colunas [ASC|DESC] -- ordena
LIMIT n;       -- limita a quantidade

SELECT * FROM membro;
SELECT nome, email FROM membro;
SELECT nome AS aluno, periodo AS sem FROM membro;
```

01:06

Este é o comando da linguagem. Tudo depois disto é variação dele.

SELECT * é ótimo para explorar e ruim em produção — diga isso agora, de passagem, sem entrar em performance.

[WHERE: filtrando linhas]

```
SELECT * FROM membro WHERE periodo > 5;
SELECT * FROM membro WHERE ativo = TRUE AND periodo >= 5;
SELECT * FROM membro WHERE periodo IN (1, 3, 5);
SELECT * FROM membro WHERE periodo BETWEEN 3 AND 7;
SELECT * FROM membro WHERE nome LIKE 'A%';
SELECT * FROM membro WHERE email LIKE '%@liga.edu.br';
SELECT * FROM membro WHERE curso_id IS NULL;
```

WHERE curso_id = NULL nunca funciona.

NULL não é comparável com =. Use IS NULL.

2026-08-27

[Capacitação de MySQL]

```
SELECT * FROM membro WHERE periodo > 5;
SELECT * FROM membro WHERE ativo = TRUE AND periodo >= 5;
SELECT * FROM membro WHERE periodo IN (1, 3, 5);
SELECT * FROM membro WHERE periodo BETWEEN 3 AND 7;
SELECT * FROM membro WHERE nome LIKE 'A%';
SELECT * FROM membro WHERE email LIKE '%@liga.edu.br';
SELECT * FROM membro WHERE curso_id IS NULL;
```

WHERE curso_id = NULL nunca funciona.
NULL não é comparável com =. Use IS NULL.

01:10 — maior fatia de prática do bloco

Peça para digitarem **um de cada vez** e olharem o resultado antes do próximo.

É aqui que a ficha cai.

LIKE: % é “qualquer coisa”, _ é “um caractere”.

Faça o erro na tela: rode WHERE curso_id = NULL, mostre que volta vazio, e só então corrija para IS NULL. Ver o erro acontecer ensina mais que o aviso.

[Ordenando e limitando]

```
SELECT nome, periodo FROM membro ORDER BY periodo DESC;

SELECT nome, periodo FROM membro
ORDER BY periodo DESC, nome ASC;

-- os 3 membros mais antigos da liga
SELECT nome FROM membro ORDER BY data_ingresso ASC LIMIT 3;
```

ASC = crescente (padrão)

DESC = decrescente

2026-08-27

[Capacitação de MySQL]

```
SELECT nome, periodo FROM membro ORDER BY periodo DESC;

SELECT nome, periodo FROM membro
ORDER BY periodo DESC, nome ASC;

-- os 3 membros mais antigos da liga
SELECT nome FROM membro ORDER BY data_ingresso ASC LIMIT 3;
```

ASC = crescente (padrão) DESC = decrescente

01:17

Mostre o critério de desempate (**ORDER BY a, b**) — é o que responde “e se dois tiverem o mesmo período?”.

LIMIT é o que evita despejar 50 mil linhas na tela.

[Exercício relâmpago]

3 minutos. Escreva o **SELECT** para:

1. Todos os membros **ativos**, mostrando só nome e e-mail
2. Membros do **1º ao 5º** período, do mais novo para o mais antigo
3. Quantos ingressaram **a partir de 2025**

Dica do 3: **SELECT COUNT(*) FROM ...**

2026-08-27

[Capacitação de MySQL]

3 minutos. Escreva o **SELECT** para:

1. **ativos**
2. **1º ao 5º**
3. **a partir de 2025**

Dica do 3: **SELECT COUNT(*) FROM ...**

01:20 — 3 min de exercício + 2 de correção

Respostas:

1. **SELECT nome, email FROM membro WHERE ativo = TRUE;**
2. **SELECT * FROM membro WHERE periodo BETWEEN 1 AND 5 ORDER BY data_ingresso DESC;**
3. **SELECT COUNT(*) FROM membro WHERE data_ingresso >= '2025-01-01';**

Corrija na tela, digitando junto. Peça para alguém ditar.

Fim do bloco 4 — devem ser 01:25.

2026-08-27

[Capacitação de MySQL]

BLOCO 5

01:25 · 10 min

BLOCO 5

Alterando e apagando

01:25 · 10 min

[UPDATE e DELETE]

```
UPDATE membro
SET    periodo = 8
WHERE  id = 1;
```

```
-- crie um registro descartavel para poder apagar com
    segurança
INSERT INTO membro (nome, email, periodo, data_ingresso)
VALUES ('Registro Teste', 'teste@liga.edu.br', 1, '2026-01-01'
);

DELETE FROM membro WHERE email = 'teste@liga.edu.br';
```

2026-08-27

[Capacitação de MySQL]

```
UPDATE membro
SET    periodo = 8
WHERE  id = 1;

-- crie um registro descartavel para poder apagar com
    segurança
INSERT INTO membro (nome, email, periodo, data_ingresso)
VALUES ('Registro Teste', 'teste@liga.edu.br', 1, '2026-01-01'
);

DELETE FROM membro WHERE email = 'teste@liga.edu.br';
```

01:25

Use o registro descartável — não deixe ninguém apagar a Elisa, que é a única sem curso e é peça do bloco 6.

A sintaxe é fácil. O próximo slide é que é o importante.

[O erro mais caro da carreira de vocês]

```
UPDATE membro SET periodo = 1; -- zera TODOS os membros
DELETE FROM membro;           -- apaga TODOS os membros
```

Sem WHERE, vale para a tabela inteira.

E não existe Ctrl+Z em SQL.

```
-- 1) veja EXATAMENTE o que sera afetado
SELECT * FROM membro WHERE id = 5;

-- 2) so entao troque SELECT * por DELETE, com o MESMO where
DELETE FROM membro WHERE id = 5;
```

2026-08-27

[Capacitação de MySQL]

```
UPDATE membro SET periodo = 1; -- zera TODOS os membros
DELETE FROM membro;           -- apaga TODOS os membros
```

Sem WHERE, vale para a tabela inteira.
E não existe Ctrl+Z em SQL.

```
-- 1) veja EXATAMENTE o que sera afetado
SELECT * FROM membro WHERE id = 5;

-- 2) so entao troque SELECT * por DELETE, com o MESMO where
DELETE FROM membro WHERE id = 5;
```

01:29 — o slide mais importante da aula

Pare. Repita. Se eles esquecerem tudo e lembrarem só disto, a capacitação valeu.

Conte um caso real, seu ou de conhecido, de **DELETE** sem **WHERE** em produção. História cola, regra não.

O MySQL Workbench tem *safe update mode* que bloqueia isso. **O phpMyAdmin não protege** — aqui a disciplina é deles.

Fim do bloco 5 — devem ser 01:35.

2026-08-27

[Capacitação de MySQL]

BLOCO 6

01:35 · 20 min

BLOCO 6

Ligando tabelas

01:35 · 20 min

[Até agora isso é uma planilha]

E se guardássemos o nome do curso dentro de **membro**?

- “Engenharia de Software” repetido em dezenas de linhas
- Alguém digita “Eng. de Software”
- Outro digita “engenharia de software”
- **A consulta por curso quebra**
- Corrigir o nome exige alterar todas as linhas

Solução: o curso vira uma tabela própria.
E **membro** guarda apenas o **id** do curso.

2026-08-27

[Capacitação de MySQL]

01:35

Retome a resposta do começo da aula — quando você perguntou “onde vocês guardam dados hoje?” e disseram planilha. É agora que se mostra o que a planilha não faz.

Este é o slide que justifica a palavra “relacional”.

E se guardássemos o nome do curso dentro de **membro**?

A consulta por curso quebra

Solução: o curso vira uma tabela própria.
E **membro** guarda apenas o **id** do curso.

[Chave primária e chave estrangeira]

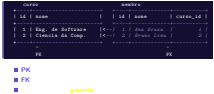
curso			membro		
id	nome		id	nome	curso_id
1	Eng. de Software	<--	1	Ana Souza	1
2	Ciencia da Comp.	<--	2	Bruno Lima	2

PK FK

- **PK** identifica a linha **dentro** da própria tabela
- **FK** **aponta** para a PK de outra tabela
- O SGBD passa a **garantir** que aquele valor existe do outro lado

2026-08-27

[Capacitação de MySQL]



01:38

O nome disso é **integridade referencial**. Diga o termo — eles vão reencontrá-lo em toda documentação.

Aponte as setas na tela. A direção importa: a FK aponta para a PK, nunca o contrário.

[Mão na massa #4: declarando a FK]

```
ALTER TABLE membro
  ADD CONSTRAINT fk_membro_curso
  FOREIGN KEY (curso_id) REFERENCES curso(id)
  ON DELETE SET NULL;
```

Agora tente inserir um curso que não existe:

```
INSERT INTO membro (nome, email, periodo, data_ingresso,
  curso_id)
VALUES ('Fake', 'fake@liga.edu.br', 1, '2026-01-01', 999);
```

ERROR 1452: a foreign key constraint fails

O erro é o **sucesso**: o banco se recusou a guardar lixo.

2026-08-27

[Capacitação de MySQL]

```
ALTER TABLE membro
  ADD CONSTRAINT fk_membro_curso
  FOREIGN KEY (curso_id) REFERENCES curso(id)
  ON DELETE SET NULL;

Agora tente inserir um curso que não existe:

INSERT INTO membro (nome, email, periodo, data_ingresso,
  curso_id)
VALUES ('Fake', 'fake@liga.edu.br', 1, '2026-01-01', 999);

ERROR 1452: a foreign key constraint fails
O erro é o sucesso: o banco se recusou a guardar lixo.
```

01:42

Faça todo mundo provocar o erro. É contraintuitivo e por isso marca: eles esperam que erro seja coisa ruim.

Frase: “sem a FK, esse 999 entrava e ninguém percebia até o relatório sair errado”.

ON DELETE SET NULL: se o curso for apagado, os membros ficam sem curso em vez de sumirem. A alternativa **CASCADE** apagaria os membros junto — cuidado com ela.

[JOIN: as duas tabelas juntas]

```
-- INNER JOIN: so os membros QUE TEM curso
SELECT m.nome AS membro, c.nome AS curso, m.periodo
FROM   membro m
INNER JOIN curso c ON m.curso_id = c.id
ORDER BY c.nome;
```

```
-- LEFT JOIN: TODOS os membros. Elisa aparece com curso NULL.
SELECT m.nome AS membro, c.nome AS curso
FROM   membro m
LEFT JOIN curso c ON m.curso_id = c.id;
```

A condição do **ON** é sempre **FK = PK**.

2026-08-27

[Capacitação de MySQL]

```
-- INNER JOIN: so os membros QUE TEM curso
SELECT m.nome AS membro, c.nome AS curso, m.periodo
FROM   membro m
INNER JOIN curso c ON m.curso_id = c.id
ORDER BY c.nome;

-- LEFT JOIN: TODOS os membros. Elisa aparece com curso NULL.
SELECT m.nome AS membro, c.nome AS curso
FROM   membro m
LEFT JOIN curso c ON m.curso_id = c.id;
```

A condição do **ON** é sempre **FK = PK**.

01:47 — o clímax da aula

Rode os dois e conte as linhas na tela: **INNER** volta 4, **LEFT** volta 5. A diferença é a Elisa. É essa comparação que ensina — não a definição.

m e **c** são apelidos. Sem eles, **nome** fica ambíguo e o MySQL responde **ERROR 1052**.

Esqueceu o ON? Produto cartesiano: cada membro cruzado com cada curso. Mostre se sobrar tempo.

[Bônus: contar e agrupar]

```
SELECT COUNT(*) FROM membro;  
SELECT AVG(periodo) FROM membro;
```

```
-- quantos membros por curso  
SELECT c.nome AS curso, COUNT(m.id) AS total  
FROM curso c  
LEFT JOIN membro m ON m.curso_id = c.id  
GROUP BY c.id, c.nome  
ORDER BY total DESC;
```

WHERE filtra linhas antes de agrupar · **HAVING** filtra grupos depois

2026-08-27

[Capacitação de MySQL]

```
SELECT COUNT(*) FROM membro;  
SELECT c.(periodo) FROM membro;  
  
-- quantos membros por curso  
SELECT c.nome AS curso, COUNT(m.id) AS total  
FROM curso c  
LEFT JOIN membro m ON m.curso_id = c.id  
GROUP BY c.id, c.nome  
ORDER BY total DESC;
```

WHERE filtra linhas antes de agrupar · **HAVING** filtra grupos depois

01:53 — SLIDE CORTÁVEL

Se o relógio passou de 01:55, pule este slide. Ele é o único conteúdo sacrificável da aula — está no guia, eles leem depois.

Se der tempo: o **LEFT JOIN** aqui é o que faz “Engenharia de Produção” aparecer com zero. Com **INNER** ela sumiria.

BLOCO 7

Fechando

01:55 · 5 min

2026-08-27

[Capacitação de MySQL]

BLOCO 7

01:55 · 5 min

[Os erros que vão acontecer com você]

- ERROR 1046** No database selected Faltou o **USE liga_SEUNOME**
- ERROR 1044** Access denied Seu banco não começa com **liga_**
- ERROR 1050** Table already exists Rodou o **CREATE TABLE** duas vezes
- ERROR 1062** Duplicate entry Violou **PRIMARY KEY** ou **UNIQUE**
- ERROR 1452** FK constraint fails Apontou para um **id** que não existe
- ERROR 1052** Column is ambiguous Faltou qualificar: **m.nome, c.nome**
- Acento virou caractere estranho Banco criado sem **utf8mb4**
- O comando não executa Faltou o **;** no fim

2026-08-27

[Capacitação de MySQL]

01:55

Não leia a tabela. Diga: “está tudo no guia, com a causa e a correção de cada um”. O valor deste slide é eles saberem que a lista existe.

ERROR 1046 No database selected Faltou o **USE liga_SEUNOME**
ERROR 1044 Access denied Seu banco não começa com **liga_**
ERROR 1050 Table already exists Rodou o **CREATE TABLE** duas vezes
ERROR 1062 Duplicate entry Violou **PRIMARY KEY** ou **UNIQUE**
ERROR 1452 FK constraint fails Apontou para um **id** que não existe
ERROR 1052 Column is ambiguous Faltou qualificar: **m.nome, c.nome**
Acento virou caractere estranho Banco criado sem **utf8mb4**
O comando não executa Faltou o **;** no fim

[Para onde ir depois]

Estudar, nesta ordem

1. Normalização (1FN, 2FN, 3FN)
2. Modelagem ER
3. Relação N:N
4. **GROUP BY** e subconsultas
5. Índices e **EXPLAIN**
6. Transações e ACID

Praticar de graça

SQLBolt
SQLZoo
DB Fiddle
HackerRank SQL

O guia completo e o script da aula
estão no repositório da capacitação.

2026-08-27

[Capacitação de MySQL]

Estudar, nesta ordem

Praticar de graça

GROUP BY
EXPLAIN

O guia completo e o script da aula
estão no repositório da capacitação.

01:57

Passe o link do repositório — no quadro e no chat da turma.

Diga que o guia tem tudo que passou aqui, mais os exercícios com resposta.
Eles não precisam ter anotado nada.

for_code[]

Obrigado!

Capacitação de MySQL

Ronivaldo D. Andrade

27 de agosto de 2026

2026-08-27

[Capacitação de MySQL]

- []

Obrigado!

Capacitação de MySQL
Ronivaldo D. Andrade
27 de agosto de 2026

02:00

Abra para perguntas enquanto eles guardam as coisas.

Não esqueça: derrubar o túnel do ngrok depois que todos saírem.